



01/11

ES6

CHEAT SHEET



Modern **JavaScript**
in one place

JS



Save this.
Refer back
every project.

LET & CONST

{ } Block **scoped** variables

...
// var – function scoped ❌

```
var name = "Mitali";
```

```
var name = "Artist"; ❌
```

// let – block scoped, reassignable ✔️

```
let age = 25;
```

```
age = 26; // allowed ✔️
```

// const – block scoped,
not reassignable ✔️

```
const city = "Pune";
```

```
city = "Mumbai"; // TypeError ❌
```

- ✔️ Always use **const** by default
- ✔️ Use **let** only when value needs to change
- ❌ Avoid **var** completely

VARIABLE COMPARISON

KEYWORD	SCOPE	REASSIGNABLE
var	Function Scoped	Yes
let	Block Scoped	Yes
const	Block Scoped	No



const
by default.
let
when needed.
Never var.

ARROW FUNCTIONS



Shorter, cleaner functions



REGULAR
FUNCTION

```
// Regular function  
function add(a, b) {  
  return a + b;  
}
```



ARROW
FUNCTION

```
// Arrow function  
const add = (a, b) => a + b;  
-----  
// Single param – no brackets needed  
const double = n => n * 2;  
-----  
// No params  
const greet = () => "Hello!";
```



Shorter syntax



Implicit return for single expressions



Inherits this from parent scope



Single
expression?
Skip the
return keyword.

TEMPLATE LITERALS



String **interpolation** done right



OLD WAY

```
const name = "Mitali";  
const role = "Developer";
```

// Old way ✗

```
const bio = "Hi, I am " + name + " a " + role;
```



TEMPLATE
LITERAL

// Template literal ✓

```
const bio = `Hi, I am ${name} a ${role}`;
```



MULTI-LINE
STRINGS

// Multi-line strings ✓

```
const card = `
```

```
  Name: ${name}
```

```
  Role: ${role}
```

```
`;
```

Clean &
readable



Backtick
replaces
quotes.

Dollar-curly
replaces **plus**.



Embed variables
with **`${}`**



Multi-line strings
without **`\n`**



Any expression
inside **`${}`**

DESTRUCTURING TURING

`</>` Extract **values** cleanly 

```
{} // Object destructuring
const user = {
  name: "Mitali",
  age: 25,
  city: "Pune"
};

const { name, age } = user;

// With rename
const { name: fullName } = user;

// With default values
const { country = "India" } = user;
```

```
[] // Array destructuring
const colors = ["pink", "teal", "yellow"];
const [primary, secondary] = colors;
```

"pink" ↓ primary (index 0)	"teal" ↓ secondary (index 1)	"yellow" ⋮ (index 2)
---	---	---



Destructuring =
unpacking values
in **one line**.

- ✓ **Cleaner** than dot notation
- ✓ **Rename** on the fly
- ✓ Works on **arrays** and **objects**

SPREAD & REST

`</>` Three dots. Two jobs.

`...` SPREAD — expand

```
const arr1 = [1, 2, 3];  
const arr2 = [...arr1, 4, 5];  
// [1, 2, 3, 4, 5]
```

```
const obj1 = { a: 1 };  
const obj2 = { ...obj1, b: 2 };  
// { a: 1, b: 2 }
```

SPREAD in Action (Expand Outward)

Array spread



Object spread



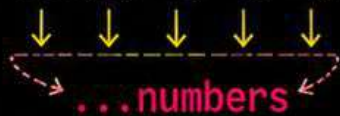
`...` REST — collect remaining

```
function sum(...numbers) {  
  return numbers.reduce((a, b) => a + b);  
}
```

```
sum(1, 2, 3, 4); // 10
```

REST in Action (Collect Inward)

`sum(1, 2, 3, 4)`



`[1, 2, 3, 4]`

(array inside function)



REMEMBER

Spread =
expand
outward.

Rest =
collect
inward.



Use **spread** to copy,
merge, or add.



Use **rest** to handle
multiple values.



Works on **arrays**,
objects, and more!

DEFAULT PARAMETERS

</> Handle **missing** arguments gracefully

✗ WITHOUT DEFAULT ✗

```
function greet(name) {  
  console.log(`Hello ${name}`);  
}  
  
greet();           // Hello undefined
```

OUTPUT

Hello **undefined**

✓ WITH DEFAULT ✓

```
function greet(name = "Guest") {  
  console.log(`Hello ${name}`);  
}  
  
greet();           // Hello Guest  
greet("Mitali");  // Hello Mitali
```

OUTPUT

Hello **Guest**

Hello **Mitali**



Set defaults
so your functions
never break
on empty input.



No more **undefined** checks



Makes functions **self-documenting**



Works with **any data type**

OPTIONAL CHAINING & ??

</> Safe access. Smart defaults.

OPTIONAL CHAINING ?.

```
// Optional chaining ?.  
const user = {  
  profile: {  
    name: "Mitali"  
  }  
};  
  
// Valid path  
console.log(user?.profile?.name);  
→ "Mitali"  
  
// Invalid path (no error)  
console.log(user?.address?.city);  
→ undefined ✓ no error
```

?? NULLISH COALESCING ??

```
// Null is replaced  
const score = null ?? 100;  
console.log(score);  
→ 100 ✓ null replaced  
  
// Falsy value is kept  
const count = 0 ?? 100;  
console.log(count);  
→ 0 ✓ keeps falsy values
```



? — stops at
null/undefined,
no crash.



?? — only falls
back on
null/undefined.

IMPORT & EXPORT

`</>` Split your code into reusable files

JS utils.js

```
// utils.js - named export
export const add = (a, b) =>
  a + b;

export const multiply = (a, b) =>
  a * b;

-----
// Default export
export default function greet() {
  // ... some code
}
```

JS main.js

```
// main.js - named import
import { add, multiply }
from "./utils.js";

// Usage
console.log(add(2, 3)); // 5
console.log(multiply(2, 4)); // 8

-----
// Default import
import greet from "./utils.js";
greet(); // Hello!
```



NAMED EXPORTS

Multiple exports
per file.

```
export { a, b, c }
```



DEFAULT EXPORT

One default export
per file.

```
export default fn
```



BENEFITS

Keeps code
organized and
reusable.



REMEMBER: Named = curly braces.

Default = no curly braces.

ASYNC AWAIT

`</>` Handle **async** code **cleanly**

PROMISE (The old way)

```
fetch("/api/user")  
  .then(res => res.json())  
  .then(data => console.log(data))  
  .catch(err => console.error(err));
```

*Callback
Hell*

ASYNC/AWAIT (The modern way) ✓ cleaner

```
async function getUser() {  
  try {  
    const res = await fetch("/api/user");  
    const data = await res.json();  
    console.log(data);  
  } catch (err) {  
    console.error(err);  
  }  
}
```

*Clean.
Flat.
Easy.*



async/await
reads like
synchronous code.



Use it.



ES6 IN ONE PLACE



FEATURE	WHAT IT DOES
<code>let / const</code>	Block scoped variables
<code>Arrow functions</code>	Shorter function syntax
<code>Template literals</code>	String interpolation
<code>Destructuring</code>	Extract values cleanly
<code>Spread / Rest</code>	Expand or collect values
<code>Default params</code>	Handle missing arguments
<code>Optional chaining</code>	Safe property access
<code>??</code>	Smart fallback values
<code>import / export</code>	Modular code
<code>async / await</code>	Clean async handling



Which ES6 feature do you use the most? 

Drop it in the comments!



Follow [@coding.artist](#)
for more JavaScript explained simply ✨